

REDROB HACKATHON · INTELLIGENT CANDIDATE DISCOVERY & RANKING

Ranking candidates the way a great recruiter actually would

Five auditable signals + a behavioral modifier + a deterministic honeypot filter — not a keyword matcher, not an LLM-per-candidate black box.

Job description: Senior AI Engineer — Founding Team, Redrob AI

Team: your-team-name-here

The problem isn't talent. It's the filter.

Recruiters go through hundreds of profiles and still miss the right person — not because the talent isn't there, but because keyword filters can't see what actually matters.

1

Keyword stuffers

Skill lists padded with AI buzzwords; title and career history don't back it up.

2

Plain-language Tier 5s

Genuinely strong builders who never wrote “RAG” or “Pinecone” on their profile.

3

Behavioral twins

Identical skills on paper, wildly different platform engagement and availability.

4

~80 honeypots

Subtly impossible profiles: “expert” in 10 skills with 0 years used, etc.

“The ‘right answer’ is not ‘find candidates whose skills section contains the most AI keywords.’ That’s a trap we’ve explicitly built into the dataset.” — [job_description.docx](#)

Five auditable signals, not one black-box score

Every score component maps to a sentence a recruiter would actually say.

30%

Semantic fit

TF-IDF cosine similarity between candidate text and the JD

25%

Skill fit

Must/nice-to-have skills, trust-weighted by endorsements + duration (not just listed)

20%

Title & role fit

Screens out keyword stuffers and off-track titles — the decisive anti-trap signal

15%

Career substance fit

Production vs. research-only, consulting-only penalty, job-hopper penalty

10%

Logistics + experience

5–9 yr band, Pune/Noida/Tier-1 India location, notice period

× behavioral modifier (recency, response rate, engagement) → final score, after a deterministic honeypot filter

Skill fit: listed $\neq$ evidenced

A skill on a list is weak evidence. A skill with endorsements and years of real use is strong evidence.

`trust(skill)`

```
= 0.3  
+ 0.5 × min(duration_months / 24,  
1)  
+ 0.2 × min(endorsements / 20, 1)
```

Must-have skill weights (embeddings, retrieval, ranking, vector DBs, NDCG/MRR, hybrid search, Python...) are multiplied by this trust score, then saturated against a realistic 4-5-strong-hit target — no real candidate has all 30 listed keywords.

Anti-stuffing penalty

If a profile lists 6+ must-have skills but their average trust score is low (typical of “I added 15 AI buzzwords yesterday”), the skill score is cut in half — directly countering keyword-stuffer traps.

Example from the dataset

A “Recommendation Systems Engineer” with FAISS, Pinecone, Embeddings, Sentence-Transformers — each with 35-88 months of real use and double-digit endorsements — scores 0.99 overall, the highest in our test sample.

Title and career history catch what the skill list can't

A

Title & role fit

- Bad-title hard floor — HR Manager, Recruiter, Marketing, Sales, Content Writer, PM — regardless of skill list
- Good-title full credit — AI/ML/NLP/search/ranking/recommendation engineer or scientist titles
- Generic “Engineer” titles get partial credit, refined by the other four signals

“A candidate who has all the AI keywords as skills but whose title is ‘Marketing Manager’ is not a fit, no matter how perfect their skill list looks.”

B

Career substance fit

- Production-language regex (deployed, shipped, scale, latency, real users) vs. research-only language
- Consulting-only penalty — fires only if every employer is TCS/Infosys/Wipro/Accenture/Cognizant/Capgemini
- Job-hopper penalty when 60%+ of roles lasted under 18 months

“If you've spent your career in pure research environments without any production deployment — we will not move forward.”

Available beats perfect-on-paper

“A perfect-on-paper candidate who hasn't logged in for 6 months and has a 5% recruiter response rate is, for hiring purposes, not actually available. Down-weight them appropriately.”

Behavioral modifier — a multiplier, not a gate

- Recency of last_active_date (35%)
- Recruiter response rate (30%)
- Open-to-work flag (15%)
- Interview completion rate (10%)
- Identity verification signals (10%)

Maps to a multiplier in [0.55, 1.15] — nudges rank, never overrides skill/role fit.

Honeypot filter — deterministic, hard-cap

- “Expert” on 3+ skills with ≤ 2 months' duration
- Total career-history months exceed stated experience by 2+ yrs
- Education end_year impossible given claimed experience

Flagged candidates are sent to the bottom of the ranking before scoring — 0 honeypots surfaced in our top-100 test run.

CPU-only, no GPU, no network — by design

An LLM-per-candidate re-ranker cannot fit a 200K-candidate production pool. TF-IDF is one input among five — used precisely because the structured signals (title, skills, career, behavior) are what actually catch the traps.

~66–90s

full 100K-candidate run, CPU only

0

GPU calls / hosted LLM calls
during ranking

0

honeypots in top-100 (test run)

valid

validate_submission.py result

5-minute / 16GB CPU budget



~90s of 300s used (≈30%)

Composite metric we're optimizing for

$$0.50 \times \text{NDCG@10} + 0.30 \times \text{NDCG@50} + 0.15 \times \text{MAP} + 0.05 \times \text{P@10}$$

Title/role + career-substance components are weighted heaviest for NDCG@10 — getting the very top results right matters 50% of the score.

One command, fully reproducible

Repository layout

rank.py

the full pipeline, single file, audit-friendly

requirements.txt

scikit-learn + numpy only

submission_metadata.yaml

team, compute, AI-tools declaration

sandbox/app.py

Streamlit small-sample demo

notebooks/eda.md

honeypot pattern notes from manual review

Reproduce in 3 lines

```
pip install -r requirements.txt
```

```
python rank.py \  
  --candidates ./candidates.jsonl \  
  --out ./submission.csv
```

```
python validate_submission.py \  
  ./submission.csv
```

No pre-computation step — rank.py is the entire pipeline, start to finish, in one process. Sandbox link required per spec §10.5.

What we'd do with more time

1

Local embedding model

Swap TF-IDF for a quantized BGE-small ONNX export at the same architectural seam, for true paraphrase-level semantic matching.

2

Overlapping-employment honeypots

Add an interval-overlap checker for concurrent “is_current” roles or overlapping date ranges — currently a TODO.

3

Company-quality weighting

Incorporate company size / known-AI-native-company lists into career_substance_fit, beyond the binary consulting-only check.

Thank you — questions welcome.